Stock Exchange Scraper: Codebase Documentation

July 31, 2025

Introduction

This document describes the structure, components, and scraping logic of the `german_scraper` codebase. The scraper is designed to efficiently download MiFID-II data from multiple German stock exchanges using Python and Playwright, with a focus on modularity, clarity, and maintainability.

1 High-Level Structure

The project is organized as a modular Python package. Below is an overview of the folder structure:

```
german_scraper/
         core/
              playwright_client.py
              utils.py
         pipelines/
              save_local.py
         exchanges/
              base.py
              berlin.py
              lsx.py
              munich.py
         cli.py
```

- **core/**: Low-level infrastructure code. Responsible for browser session management and general-purpose utility functions. Used by all scrapers.
- **pipelines/**: Implements logic for storing or post-processing downloaded files (e.g., saving locally, uploading to S3, etc.).
- **exchanges/**: Contains one Python module per exchange. Each implements scraping logic and selectors specific to that exchange, in a reusable class.
- **cli.py**: The main script that acts as the program's entry point. Orchestrates the scraping process across all exchanges.

2 Folder and File Descriptions

1. core/

playwright_client.py:
- Defines `PlaywrightClient`, a helper class that encapsulates launching and closing a Playwright browser instance (Chromium).
- Provides an async context manager (`launch()`) so scrapers can share a single browser instance efficiently.
- Ensures resources are cleaned up and that each scraper gets a new page object.

utils.py:
- Includes reusable functions such as:
  - `random_delay(min, max)`: Asynchronously sleeps for a random duration, used to simulate human-like delays between downloads.
  - `click_first_consent(page)`: Detects and clicks cookie-consent or privacy popups, supporting different wordings (e.g., Accept, Weiter, OK).
- These utilities are used by multiple scrapers for consistency and code cleanliness.

2. pipelines/

save_local.py:
- Implements `SaveLocalPipeline`, a class that handles writing downloaded files to a local directory, structured by exchange and file type.
- The `save(download, subdir)` method:
  - Creates target folders if needed (`downloads/exchange/`).
  - Checks for existing files to avoid redundant downloads.
  - Saves each file with its original name for traceability.
  - Returns the path for logging or further processing.
- Additional pipelines (e.g., cloud upload) can be added for scalability.

3. exchanges/

base.py:
- Defines the abstract base class `Exchange`, which enforces the interface for all scraper modules (must implement a `run()` method).
- Stores references to the shared browser and pipeline objects.
- Provides an extensible architecture, so adding new exchanges only requires creating a new file inheriting from `Exchange`.

berlin.py, lsx.py, munich.py:
- Each file contains a class (`Berlin`, `LSX`, `Munich`) that implements all scraping logic for a specific exchange.
- Follows the pattern:
  - Opens a new page in the shared browser session.
  - Navigates to the exchange's download URL(s).
  - Handles any cookie consent using `click_first_consent`.

  - Locates all relevant download links using CSS or role selectors that mirror those in the original, working Playwright scripts (ensuring compatibility and reliability).
  - For each link:
    * In debug mode: prints what would be downloaded, without triggering network requests.
    * In production mode: clicks the link, listens for the download, and invokes the pipeline to save the file.
  - Ensures the browser page is closed after processing.
- This approach keeps logic for each exchange modular and easy to update as websites change.

4. cli.py

- This is the **main entry point** of the project.
- Responsible for:
  - Initializing the download pipeline (`SaveLocalPipeline`).
  - Launching a shared Playwright browser using `PlaywrightClient`.
  - Creating an instance of each exchange scraper and passing them the shared browser and pipeline.
  - Running all scrapers in sequence (`for s in scrapers: ...`).
  - Wrapping each scraper in a `try/except` block to ensure that an error in one exchange does not affect the others (robustness).
  - Providing a `debug` flag so that the scraping process can be simulated quickly and safely (no real downloads).
  - Closing the browser and cleaning up resources at the end of execution.
- Designed for both production (downloading real data) and testing/dry-run workflows.

3 Detailed Logic of the Scraping Process

1. **Initialization:**
   - Start the Playwright browser via `PlaywrightClient`.
   - Create an instance of the output pipeline (e.g., `SaveLocalPipeline`).
2. **Scraper Instantiation:**
   - For each exchange (e.g., Berlin, LSX, Munich), instantiate a class inheriting from `Exchange`.
   - Each scraper receives the shared browser and pipeline.
3. **Scraping Workflow (per exchange):**

- Open a new page and navigate to the target exchange's data download site.
- Use shared helpers (e.g., `click_first_consent`) to accept cookie banners if present.
- Locate download links using selectors mirroring those from the original, working Playwright scripts (e.g., matching anchor text with regex).
- In debug mode, print all found file links and simulate actions with small delays.
- In real mode, for each link:
  - Use Playwright's `expect_download` context to listen for a file download after clicking the link.
  - Save the file with the pipeline (typically into `downloads/exchange/`).
  - Wait a random interval before continuing to the next file.
- Close the page after processing.

4. **Error Handling:**
   - Each scraper runs inside a `try/except` block in `cli.py`, so failures in one exchange do not prevent the others from running.
5. **Termination:**
   - After all scrapers have finished, close the browser and exit.

4 How to Run

Virtual Environment Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install playwright rich
python -m playwright install
```

Running the Scraper

```
python -m german_scraper.cli --debug # Dry-run (recommended for testing)
python -m german_scraper.cli        # Real download mode
```

5 Extending the Scraper

- Add a new exchange by creating a new file in `exchanges/` that implements a subclass of `Exchange`.
- If you wish to save data elsewhere (e.g., S3, database), add a new pipeline in `pipelines/` and pass it to the scrapers.

6 Conclusion

This modular approach ensures the scraping project is robust, easy to extend, and straightforward for multiple developers to maintain. All selectors and scraping steps are clearly separated per exchange, and changing the download/output method is as simple as swapping the pipeline.